

TinyURL — Medium Scale System Design

1. Functional Requirements

- Create a short URL from a long URL
- Redirect users to the original URL
- Allow custom aliases
- Allow URL expiration
- Let users disable their own links
- Track basic click count
- Reject invalid or unsafe URLs

2. Non-Functional Requirements

- Availability: 99.9%
- Redirect latency: <150 ms for cached links
- Peak traffic: ~8,000 requests/sec
- Durable storage with backups
- Unique short codes
- HTTPS and basic abuse protection

3. Back-of-the-Envelope Estimation

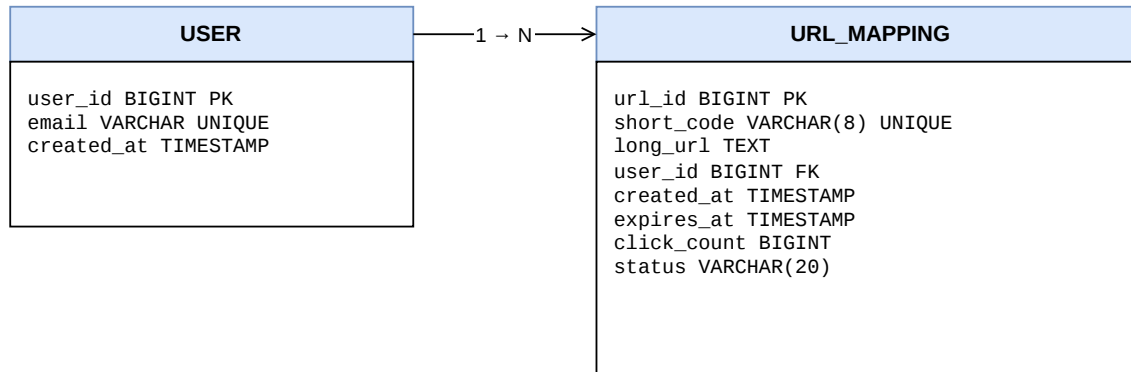
Assumptions: 2 million new URLs/day, 300 million redirects/day, average mapping size 700 bytes, 3 years retention

Metric	Estimate
New URLs/day	2 million
Redirects/day	300 million
Average read QPS	~3,500
Peak QPS	~8,000
Storage/year	~0.5 TB
Storage/3 years	~1.5 TB

Design choice: traffic is read-heavy, so a cache is placed before the database.

Database & API Design

4. Database Design (PostgreSQL)



Indexes: UNIQUE(short_code), INDEX(user_id), INDEX(expires_at)

5. API Design

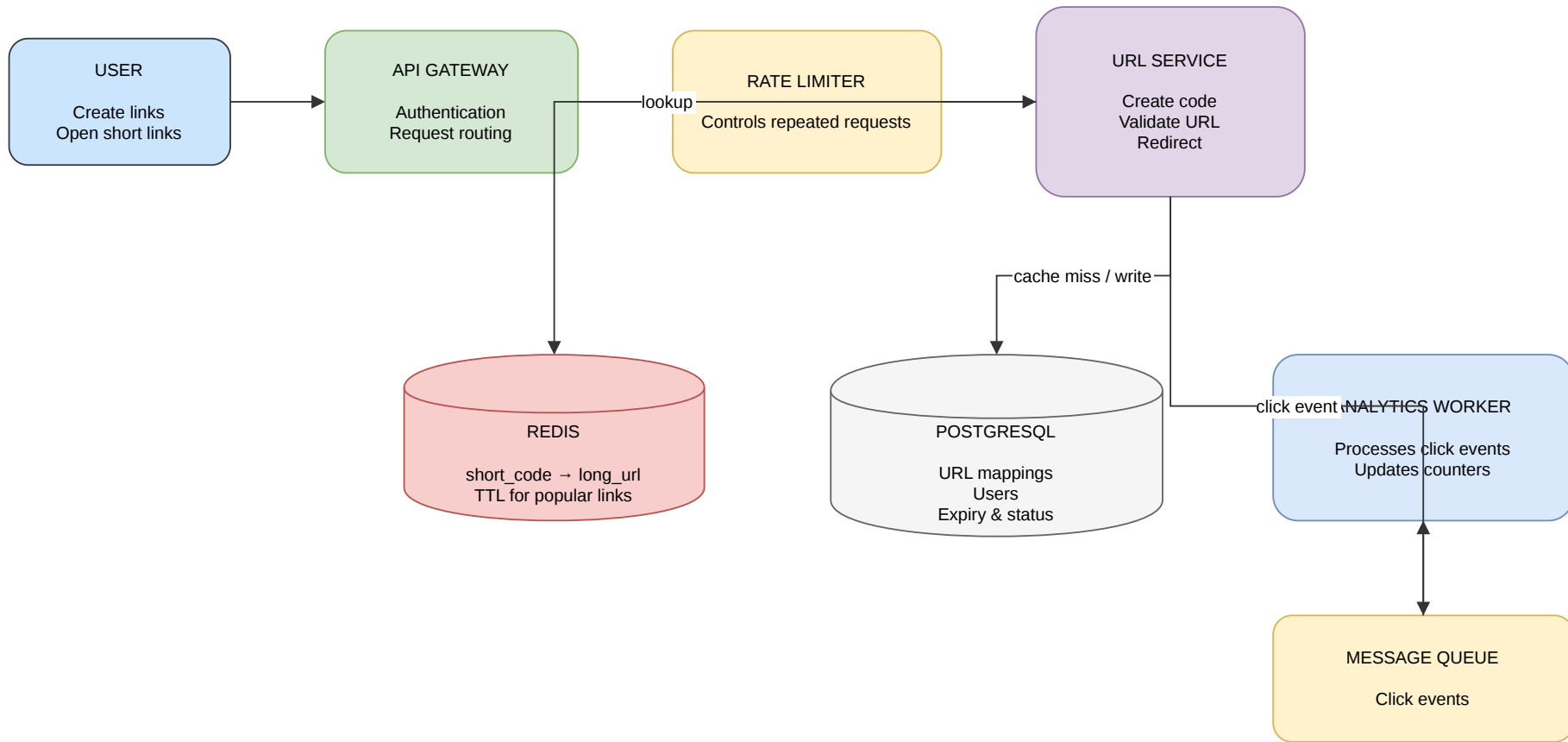
Method	Endpoint	Purpose
POST	/api/v1/links	Create short URL
GET	/api/v1/links/{code}	View link metadata
GET	/{code}	Redirect to long URL
PATCH	/api/v1/links/{code}	Disable or update expiry
GET	/api/v1/links/{code}/stats	Get click statistics

Example

```
POST /api/v1/links
{ "longUrl": "https://example.com/projects/system-design/notes" }
```

Response:
<https://turl.app/k7P2xQ>

TinyURL — High Level Architecture



Create URL: validate → generate ID → Base62 → PostgreSQL → Redis

Redirect: Redis hit → redirect; miss → PostgreSQL → populate Redis → redirect